

yolo_final 项目汇报辅助材料

第一部分：模型架构模块介绍

2026 年5 月6 日

本部分汇报目标

本部分用于介绍yolo_final 当前实验线中检测模型的核心结构。汇报重点不是泛泛介绍YOLO，而是说明我们当前代码中实际使用的网络模块、张量流向、检测结果生成方式以及训练loss 的组成。需要讲清楚的内容包括：

1. yolo_final 的网络架构；
2. yolo_final 的backbone 结构；
3. backbone 中使用的小模块；
4. 检测器整体结构；
5. decoupled head 的结构；
6. prediction tensors 到最终detection results 的转换；
7. 最终训练使用的loss 结构，以及每一个term 的具体含义。

当前主线实验采用COCO-only 训练评估，输入分辨率为 416×416 ，模型配置为deep_residual_dual_scale，使用P4/P5 双尺度输出、每尺度3 个anchor、轻量FPN/PAN neck 和decoupled detection head。P3 是后续扩展实验方向，本部分先以当前P4/P5 双尺度主线模型为核心对象。

Table 1: 当前主线模型配置

项目	当前配置
输入分辨率	416×416
Backbone	deep_residual_dual_scale_three_box
Feature levels	p4,p5
Neck	fpn_pan_lite
Head	decoupled
每尺度anchor 数	3
类别数	COCO 80 类
训练数据	COCO train only

1 yolo_final 网络架构

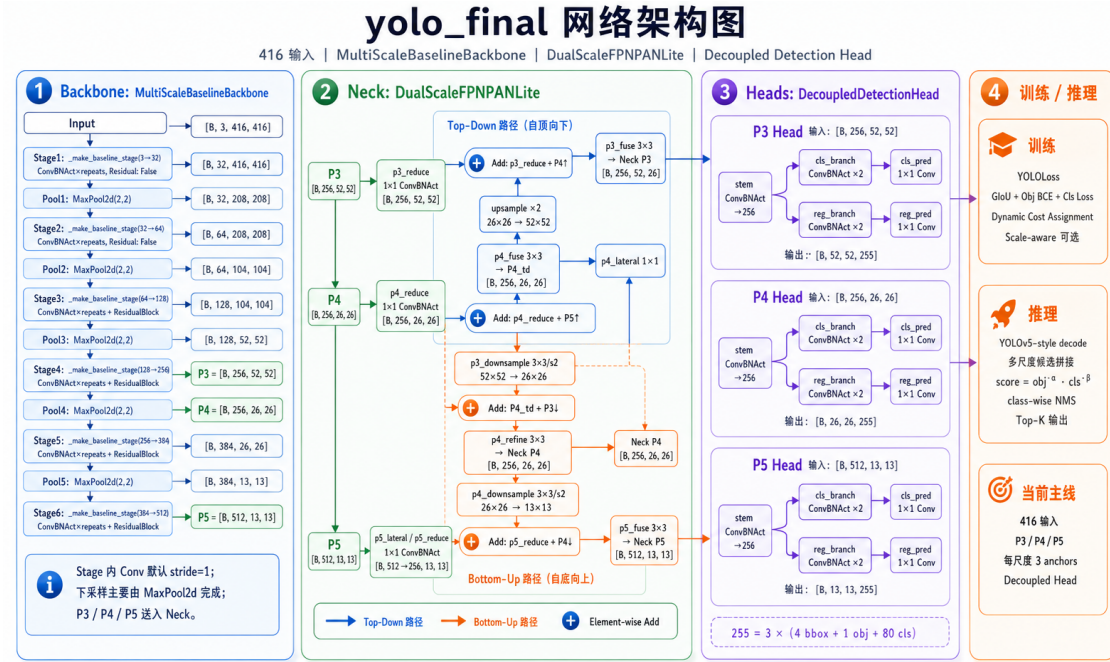


Figure 1: yolo_final 网络架构示意图

yolo_final 当前主线是一个YOLO-style dense detector。整体流程可以概括为:

$$I \rightarrow B(I) \rightarrow N(P_4, P_5) \rightarrow H_4, H_5 \rightarrow \hat{Y}.$$

其中, I 表示输入图像, $B(\cdot)$ 表示backbone, $N(\cdot)$ 表示neck, H_4, H_5 分别表示P4 和P5 上的检测头, $\hat{Y}$ 表示多尺度prediction tensors。

从工程实现上看, 模型由四个核心部分组成:

- Backbone: 将输入图像转换为不同空间分辨率和不同语义层级的特征图;
- Neck: 使用轻量FPN/PAN 路径融合P4/P5 特征;
- Decoupled Head: 每个尺度独立输出box、objectness 和class logits;
- Loss/Decoder: 训练阶段进入YOLOLoss, 推理阶段进入decode、score filtering 和NMS。

当前模型输出两个尺度的prediction tensors:

$$\hat{Y}_{p4} \in \mathbb{R}^{B \times 26 \times 26 \times 255}, \quad \hat{Y}_{p5} \in \mathbb{R}^{B \times 13 \times 13 \times 255}.$$

这里 $255 = 3 \times (4 + 1 + 80)$ 。其中3 表示每个grid cell 有3 个anchor; 4 表示box 参数; 1 表示objectness; 80 表示COCO 的类别logits。

Table 2: 整体网络的张量接口

模块	输入	输出	作用
Backbone	$[B, 3, 416, 416]$	P_4, P_5	提取多尺度视觉特征
Neck	P_4, P_5	$\tilde{P}_4, \tilde{P}_5$	融合语义与定位信息
Head-P4	$[B, C_4, 26, 26]$	$[B, 26, 26, 255]$	中等尺度目标预测
Head-P5	$[B, C_5, 13, 13]$	$[B, 13, 13, 255]$	大目标与强语义目标预测
Decoder	多尺度logits	box/score/class	生成检测结果

2 Backbone 结构

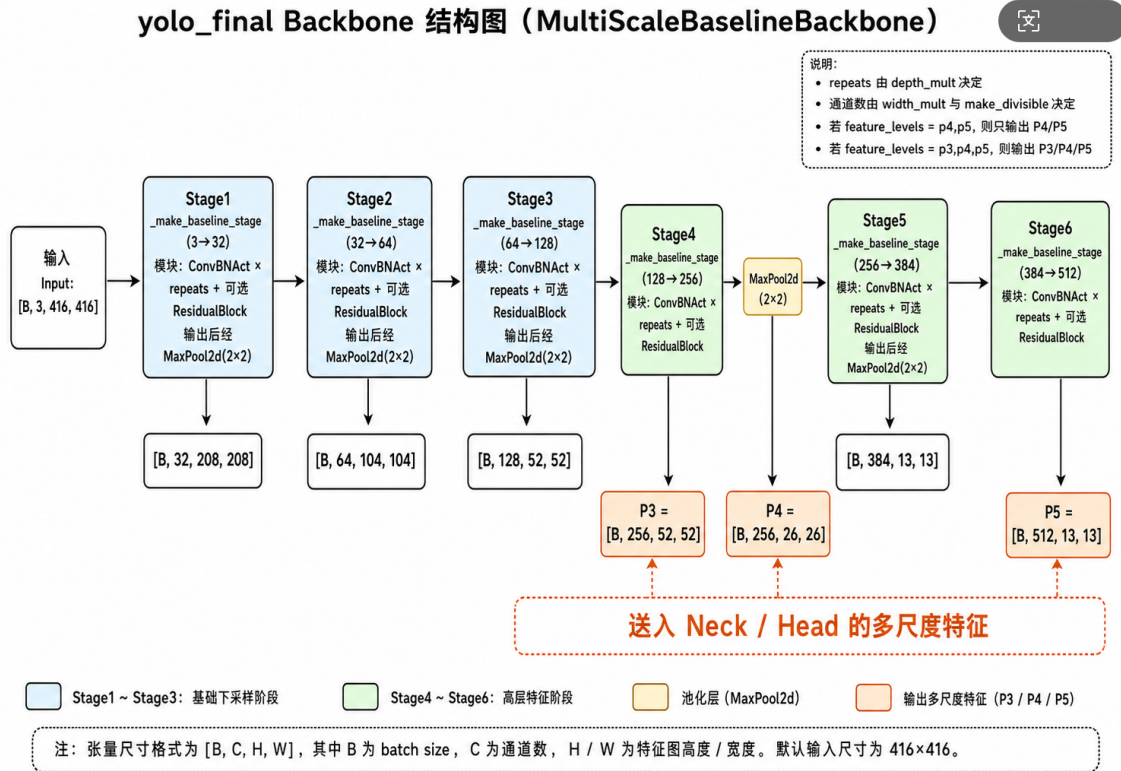


Figure 2: Backbone 结构示意图

当前主线使用的backbone 是MultiScaleBaselineBackbone。它由6个stage组成,通过卷积模块提取特征,并通过MaxPool完成空间下采样。前两个stage不使用残差模块,主要负责浅层纹理、边缘和局部组合模式的提取;从stage3开始加入ResidualBlock,用于提升中后层表达能力并改善深层训练稳定性。

当输入图像尺寸为 416×416 时,当前主线主要使用两个检测尺度:

- P_4 : 约 26×26 , 通道数为256, 主要负责中等尺度目标;
- P_5 : 约 13×13 , 通道数为512, 主要负责更大目标和更强语义目标。

在后续P3扩展实验中,模型会额外输出 52×52 的P3特征,用于增强小目标召回。但当前汇报的主线结构仍以P4/P5双尺度为核心。

当前配置中depth_mult=2.0。每个stage内部基础卷积重复数为:

$$\text{repeat} = \max(1, \text{round}(2 \times \text{depth_mult})) = 4.$$

Table 3: Backbone stage 设计

Stage	通道数	是否residual	作用说明
Stage1	32	否	提取浅层纹理，随后下采样
Stage2	64	否	提取边缘和局部组合模式
Stage3	128	是	开始引入残差增强
Stage4	256	是	产生P3 候选特征，当前主线经pool 得到P4
Stage5	384	是	聚合更强上下文语义
Stage6	512	是	产生P5 高语义特征

因此，每个stage 不是单层卷积，而是由多层ConvBNAct 加可选ResidualBlock 构成。

3 Backbone 小模块示意图

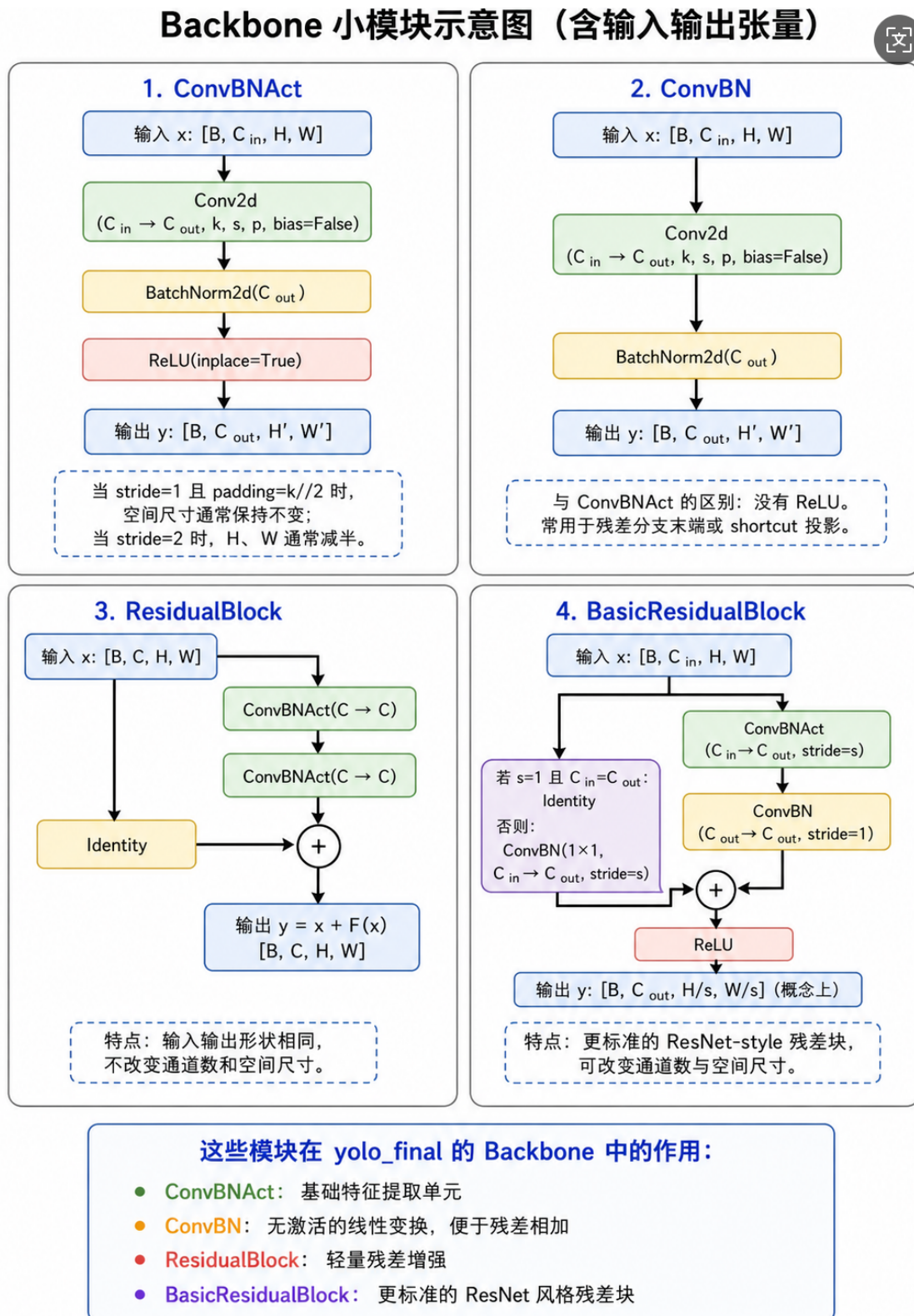


Figure 3: Backbone 小模块示意图

Backbone 中主要涉及三个基础模块: ConvBNAct、ConvBN 和 ResidualBlock。代码中也保留了 BasicResidualBlock, 但当前主线没有使用 ResNet18-like backbone, 因此它不是当前实验线的实际主干模块。

3.1 ConvBNAct

ConvBNAct 是最基础的特征提取模块，结构为：

$$\text{Conv2d} \rightarrow \text{BatchNorm2d} \rightarrow \text{ReLU}.$$

该模块负责局部感受野建模、通道变换和非线性表达，是backbone、neck 和head 中最常用的基础单元。

3.2 ConvBN

ConvBN 的结构为：

$$\text{Conv2d} \rightarrow \text{BatchNorm2d}.$$

它不带激活函数，主要用于残差分支末端或shortcut 对齐。保留无激活输出可以避免在残差相加前破坏线性残差路径。

3.3 ResidualBlock

当前主线实际使用的是轻量ResidualBlock：

$$F(x) = \text{ConvBNAct}_{3 \times 3}(\text{ConvBNAct}_{3 \times 3}(x)), \quad y = x + F(x).$$

该模块要求输入输出通道一致，不负责下采样，主要作用是在同一空间尺度上进一步细化特征。它可以缓解深层堆叠带来的优化难度，使中后层特征学习更加稳定。

3.4 BasicResidualBlock

BasicResidualBlock 是代码中保留的ResNet 风格模块，支持stride 下采样和shortcut 通道对齐：

$$y = \text{ReLU}(F(x) + S(x)).$$

需要强调的是：当前实验线并不是没有残差块。从stage3 到stage6 都启用了ResidualBlock；只是当前主线没有使用标准ResNet 的BasicResidualBlock 作为backbone。

4 检测器整体结构

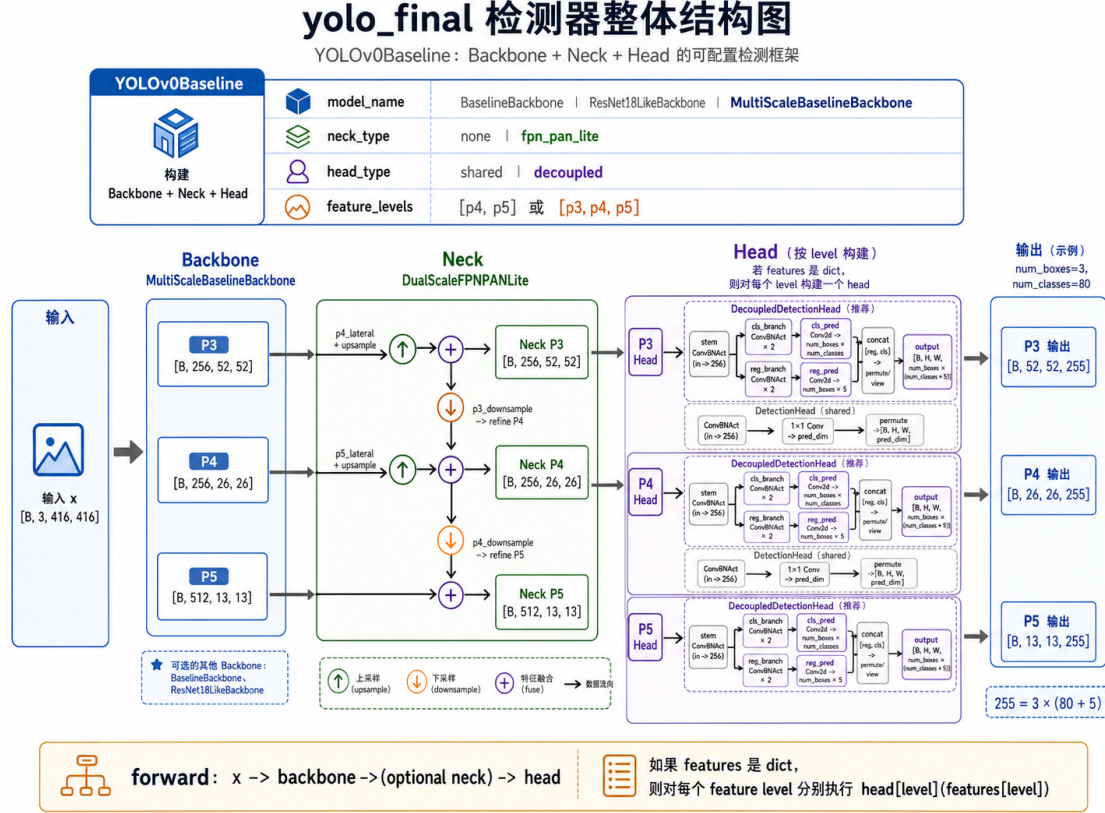


Figure 4: yolo_final 检测器整体结构

检测器由YOLOv0Baseline 作为统一入口进行构建。该类不是单一固定网络，而是根据配置动态组装backbone、neck 和head:

- `_build_backbone`: 根据`model.name` 选择backbone;
- `_build_neck`: 根据`neck_type` 决定是否插入FPN/PAN Lite;
- `_build_head`: 根据`head_type` 选择shared head 或decoupled head。

当前主线中，backbone 和neck 输出的是一个多尺度字典:

$$\{p4 : \tilde{P}_4, p5 : \tilde{P}_5\}.$$

head 使用ModuleDict，对每个尺度单独建立检测头，最终输出:

$$\{p4 : \hat{Y}_{p4}, p5 : \hat{Y}_{p5}\}.$$

4.1 Neck: 轻量FPN/PAN 融合

当前neck 为DualScaleFPNPANLite。它的作用是让P4 获得更强的高层语义，同时让P5 补充更细的定位信息。

Top-down 路径中，P5 经过 1×1 lateral projection 后上采样，与P4 对齐相加，再经过 3×3 卷积融合:

$$P_4^{td} = \text{Conv}_{3 \times 3} (\text{Conv}_{1 \times 1}(P_4) + \text{Up}(\text{Conv}_{1 \times 1}(P_5))) .$$

Bottom-up 路径中，融合后的P4 再下采样回P5 尺度，与原始P5 投影结果相加：

$$P_5^{out} = \text{Conv}_{3 \times 3} (\text{Conv}_{1 \times 1}(P_5) + \text{Down}(P_4^{out})) .$$

相对完整FPN/PAN，当前版本更轻量，适合在我们当前YOLOv1 实验框架中做可控对比。

5 Decoupled Head 的结构介绍

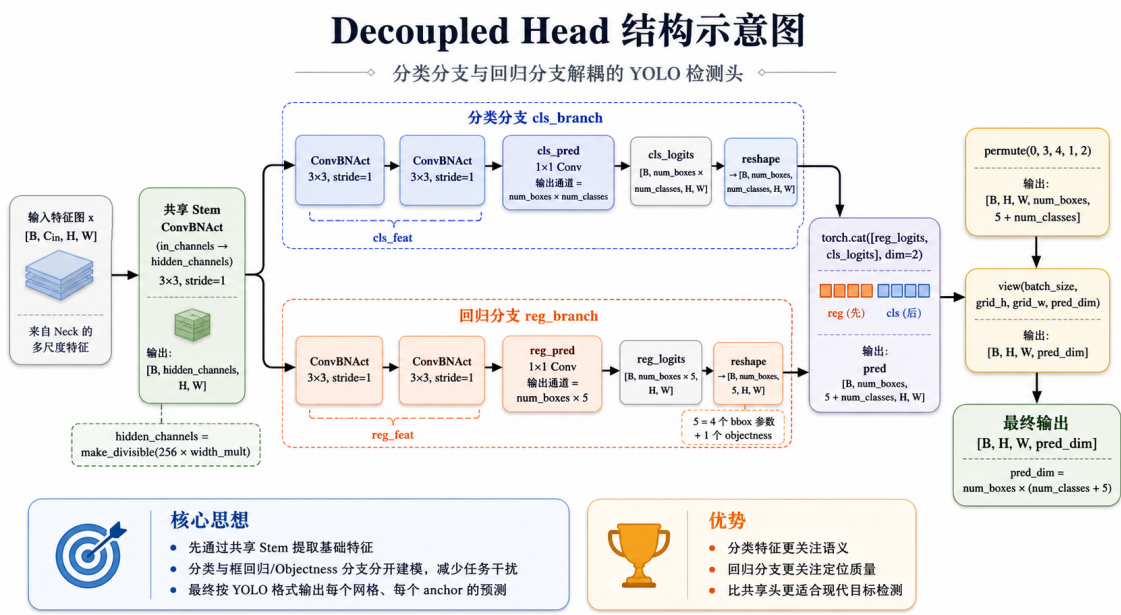


Figure 5: Decoupled Detection Head 结构示意图

当前主线检测头为DecoupledDetectionHead。其核心思想是：分类任务和定位任务虽然共享输入特征，但在最后的预测分支中分开建模。

对任一尺度 s ，输入特征为：

$$X_s \in \mathbb{R}^{B \times C_s \times H_s \times W_s} .$$

首先经过shared stem:

$$X'_s = \text{ConvBNAct}_{3 \times 3}(X_s) .$$

然后进入两个独立分支：

$$Z_s^{cls} = \text{Conv}_{1 \times 1} (\text{ClsBranch}(X'_s)) \in \mathbb{R}^{B \times (A \cdot C) \times H_s \times W_s} ,$$

$$Z_s^{reg} = \text{Conv}_{1 \times 1} (\text{RegBranch}(X'_s)) \in \mathbb{R}^{B \times (A \cdot 5) \times H_s \times W_s} .$$

其中，classification branch 由两层ConvBNAct 构成，输出 $A \times C$ 个类别logits； regression branch 同样由两层ConvBNAct 构成，输出 $A \times 5$ 个box/objectness logits。

每个anchor 的输出为:

$$(t_x, t_y, t_w, t_h, t_o, \ell_1, \ell_2, \dots, \ell_{80}).$$

最终reshape 和concat 后得到:

$$\hat{Y}_s \in \mathbb{R}^{B \times H_s \times W_s \times A(5+C)}.$$

在当前COCO 主线中, $A = 3, C = 80$, 因此:

$$\hat{Y}_{p4} \in \mathbb{R}^{B \times 26 \times 26 \times 255}, \quad \hat{Y}_{p5} \in \mathbb{R}^{B \times 13 \times 13 \times 255}.$$

Decoupled head 的优势在于: 分类更关注语义可分性, 回归更关注几何定位和objectness, 两者不再完全共享最后的预测特征, 有利于降低任务冲突。

6 Prediction Tensors 到Detection Results

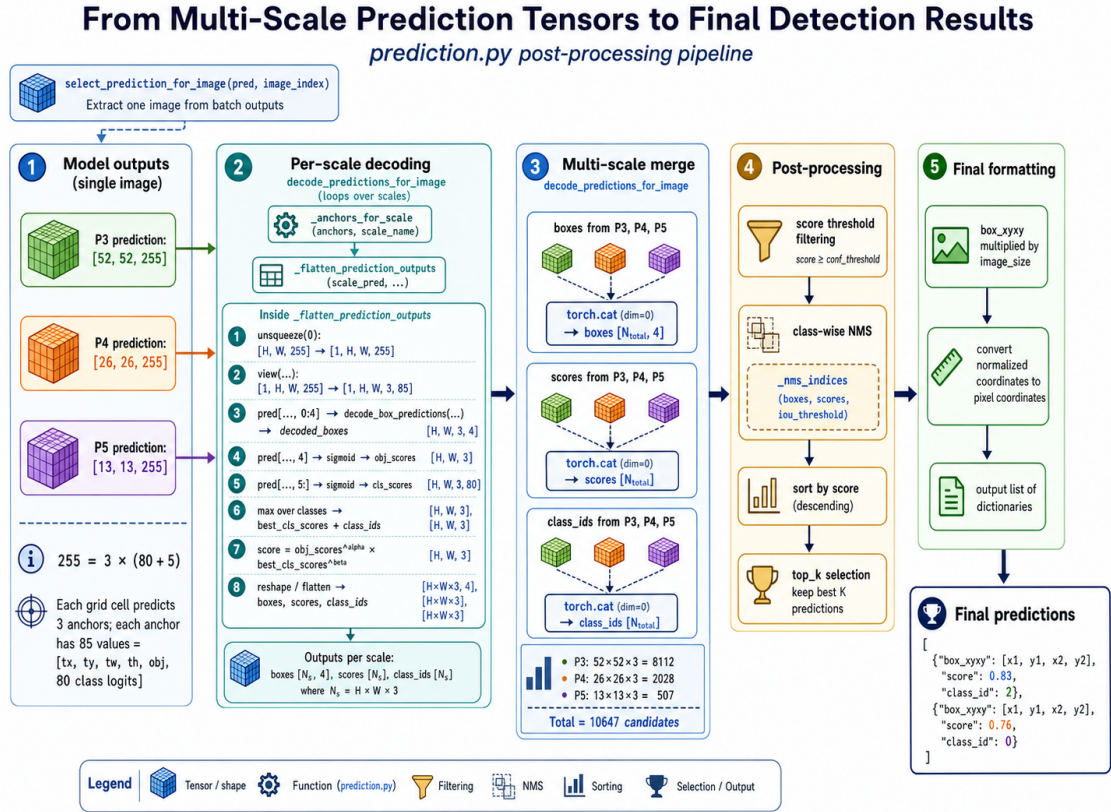


Figure 6: Prediction Tensors 到Detection Results 的转换流程

推理阶段, 模型输出的prediction tensors 还不是最终检测结果。它们需要经过decode、score 计算、阈值过滤和NMS, 才能转换为最终的detection results。

整体流程如下:

1. 按尺度取出 $\hat{Y}_{p4}$ 和 $\hat{Y}_{p5}$;
2. 将每个尺度的输出reshape 为 $[H, W, A, 85]$;
3. 根据anchor 和YOLOv5 box parameterization 解码box;

4. 对objectness logit 和class logits 做sigmoid;
5. 计算每个anchor 的最终score;
6. 按score threshold 过滤低置信度预测;
7. 对每个类别执行class-wise NMS;
8. 按score 降序排序并保留top-k;
9. 输出box、score 和class id。

6.1 Score 计算

每个anchor 的类别分数由class logits 得到:

$$c^* = \arg \max_c \sigma(\ell_c), \quad p_{cls} = \max_c \sigma(\ell_c).$$

最终排序分数为:

$$score = \sigma(t_o)^\alpha \cdot p_{cls}^\beta.$$

当前评估和可视化中通常使用 $\alpha = 2.0, \beta = 1.0$, 也就是说objectness 被赋予更强的影响。如果objectness 校准不好, 最终排序会受到明显影响, 因此我们后续也做过score sweep 来检查score calibration。

6.2 最终检测结果格式

最终每个detection result 是一个字典:

$$\{\text{box_xyxy}, \text{score}, \text{class_id}\}.$$

其中, `box_xyxy` 表示左上角和右下角坐标。模型内部先在resize 后的 416×416 坐标系中得到框; 正式COCO 评估时, 会再映射回原始图像坐标。

7 最终使用的Loss 结构



Figure 7: YOLOLoss 结构示意图

当前训练使用的loss 由三个主要部分组成: box regression loss、objectness loss 和classification loss。对于每个尺度 $s \in \{p4, p5\}$:

$$\mathcal{L}^{(s)} = \lambda_{box} \mathcal{L}_{box}^{(s)} + \lambda_{obj} \mathcal{L}_{obj}^{(s)} + \lambda_{cls} \mathcal{L}_{cls}^{(s)}.$$

多尺度总loss 为:

$$\mathcal{L}_{total} = \sum_s w_s \mathcal{L}^{(s)}.$$

当前主线配置中:

$$\lambda_{box} = 5, \quad \lambda_{obj} = 1, \quad \lambda_{cls} = 1, \quad \lambda_{noobj} = 1.$$

7.1 Loss 前置步骤: Dynamic Assignment

训练时, 网络输出的是dense predictions, 即所有grid cell、所有anchor 上的预测槽位; 而每张图中的GT box 数量远少于预测槽位。因此, 在计算loss 之前, 必须先决定每个GT 由哪些预测槽位负责监督。

当前主线使用dynamic_cost assignment。对于每个GT, 先在GT 中心附近选择候选slots, 然后计算匹配代价:

$$\text{Cost} = \lambda_{box}^{cost}(1 - \text{IoU}) + \lambda_{cls}^{cost}\text{BCE}_{cls} + \lambda_{anchor}^{cost}\text{ShapeCost}.$$

其中:

- $(1 - \text{IoU})$ 表示预测框与GT 框的几何不匹配程度;
- BCE_{cls} 表示当前预测类别与GT 类别之间的分类代价;
- ShapeCost 是可选anchor shape prior, 用于衡量anchor 宽高与GT 宽高的匹配程度;
- 主线基础配置中 $\lambda_{anchor}^{cost} = 0$, 后续anchor-cost 实验中测试过非零形状先验。

每个GT 根据cost 选择最多`dynamic_topk=2` 个正样本。正样本参与box loss、objectness positive loss 和classification loss; 负样本参与objectness negative loss; 高IoU 但没有被分配为正样本的预测会被ignore, 不作为背景惩罚。

7.2 Box Regression Loss

Box regression loss 用于监督预测框的位置和尺度。当前使用GIoU 形式:

$$\mathcal{L}_{box} = \frac{1}{N_{pos}} \sum_{i \in Pos} (1 - \text{GIoU}(b_i, \hat{b}_i)).$$

其中, N_{pos} 表示正样本数量, b_i 表示预测框, $\hat{b}_i$ 表示对应GT 框。

该项的物理意义是: 让预测框与GT 框在空间上尽可能重合。IoU 衡量预测框与GT 框的重叠比例; GIoU 在两个框不相交时仍然能提供有效惩罚, 因此比普通IoU loss 更适合训练早期的框回归。

该loss 只在positive samples 上计算, 不惩罚背景位置的box。当前 $\lambda_{box} = 5$, 说明定位质量是总loss 中权重最高的监督项。

7.3 Objectness Loss

Objectness loss 用于判断某个预测槽位是否负责一个真实目标。当前objectness 被拆成正样本项和负样本项。

对正样本, objectness target 使用IoU soft target:

$$y_{obj}^+ = \max(\text{IoU}(b_i, \hat{b}_i), \tau), \quad \tau = 0.05.$$

对应loss 为:

$$\mathcal{L}_{obj}^+ = \frac{1}{N_{pos}} \sum_{i \in Pos} \text{BCEWithLogits}(o_i, y_{obj}^+).$$

这里使用IoU soft target 的原因是: objectness 不只表示“这里有没有目标”, 也隐含表示“这个预测框是否高质量”。如果一个正样本框与GT 的IoU 很低, 它不应该被训练成过高的objectness。

对负样本, target 为0:

$$\mathcal{L}_{obj}^- = \frac{1}{N_{neg}} \sum_{i \in Neg} \text{BCEWithLogits}(o_i, 0).$$

最终objectness loss 为:

$$\mathcal{L}_{obj} = \mathcal{L}_{obj}^+ + \lambda_{noobj} \mathcal{L}_{obj}^-.$$

我们将 λ_{noobj} 从早期的0.5 调整到1.0, 目的是增强背景抑制, 降低误检候选框过多的问题。

7.4 Classification Loss

当前分类分支使用`quality_bce` 语义。分类目标不是简单one-hot, 而是质量感知soft target:

$$y_{cls,c} = \begin{cases} \text{IoU}(b_i, \hat{b}_i), & c = c^*, \\ 0, & c \neq c^*. \end{cases}$$

分类loss 为:

$$\mathcal{L}_{cls} = \frac{1}{N_{pos}C} \sum_{i \in Pos} \sum_{c=1}^C \text{BCEWithLogits}(\ell_{i,c}, y_{cls,c}).$$

该项的意义是: 分类置信度不仅表示目标属于哪个类别, 还受到定位质量约束。低IoU 的匹配不会被强行训练成高分类置信度, 有助于让分类分数和框质量更一致。

7.5 Loss 监控指标

训练过程中除了记录总loss, 还需要记录各个子项和assignment 指标。它们用于判断训练是否正常, 但不等价于最终COCO AP。

Table 4: Loss 相关监控指标

指标	含义	用途
<code>loss_box</code>	GIoU 框回归损失	判断定位学习是否改善
<code>loss_obj</code>	objectness 总损失	判断前景/背景置信度学习情况
<code>loss_obj_pos</code>	正样本objectness loss	看正样本置信度是否学起
<code>loss_obj_neg</code>	负样本objectness loss	看背景抑制是否足够
<code>loss_cls</code>	分类BCE loss	判断类别分支学习情况
<code>mean_giou</code>	正样本平均GIoU	定位质量的训练侧指标
<code>mean_obj_target</code>	正样本objectness target 均值	检查IoU soft target 状态
<code>mean_cls_target</code>	分类soft target 均值	检查quality target 状态
<code>positive_cells_per_image</code>	每图正样本槽位数	检查assignment 是否异常
<code>collision_count</code>	GT 落入同一cell 的冲突数	检查dense assignment 压力
<code>ignored_count</code>	ignore 槽位数	检查高IoU 非正样本过滤情况
<code>dropped_gt_count</code>	未分配GT 数	检查GT 是否被漏分配

8 本部分小结

`yolo_final` 当前主线是一个配置驱动的YOLO-style dense detector, 其结构可以概括为:

Backbone + FPN/PAN Lite + Decoupled MultiScale Heads + YOLOLoss/Decoder.

当前模型的关键特点包括:

- 使用P4/P5 双尺度，兼顾中等目标和大目标；
- 每尺度3 anchors；不同实验可使用原始anchors 或COCO train GT box 重新聚类anchors；
- backbone 中实际使用残差结构，但不是标准ResNet backbone；
- 使用decoupled head，降低分类任务和定位任务在最终预测层的冲突；
- 使用GIoU box loss、objectness BCE 和quality-aware classification BCE；
- 训练侧通过dynamic assignment 生成正负样本，推理侧通过decode、score filtering 和class-wise NMS 生成最终检测结果。

后续实验中的提高分辨率、重新聚类anchors、增加P3、调整assignment 和score calibration，本质上都围绕三个问题展开：特征尺度是否覆盖足够、候选框质量是否稳定、最终分数排序是否可靠。

9 第二部分：一次完整实验的流程与结果

本部分选取实验

dual_scale_three_box_coco_only_noobj1_416_ddp_20260430_124818

作为完整案例，说明一次正式实验从配置、训练、保存、可视化、评估到指标解释的全流程。该实验是COCO-only、P4/P5 双尺度、每尺度3 anchor、原始P4/P5 anchor、 $\lambda_{noobj} = 1.0$ 的8 GPU DDP 训练实验，也是当前正式COCO AP、AP50、AP75 综合最好的结果展示轮次。

9.1 实验结果展示

训练脚本在实验结束后默认保存4 张验证集可视化结果。这4 张图主要用于快速sanity check: 观察模型是否能输出框、类别是否大致合理、置信度是否过低或过高、是否存在明显候选框爆炸。

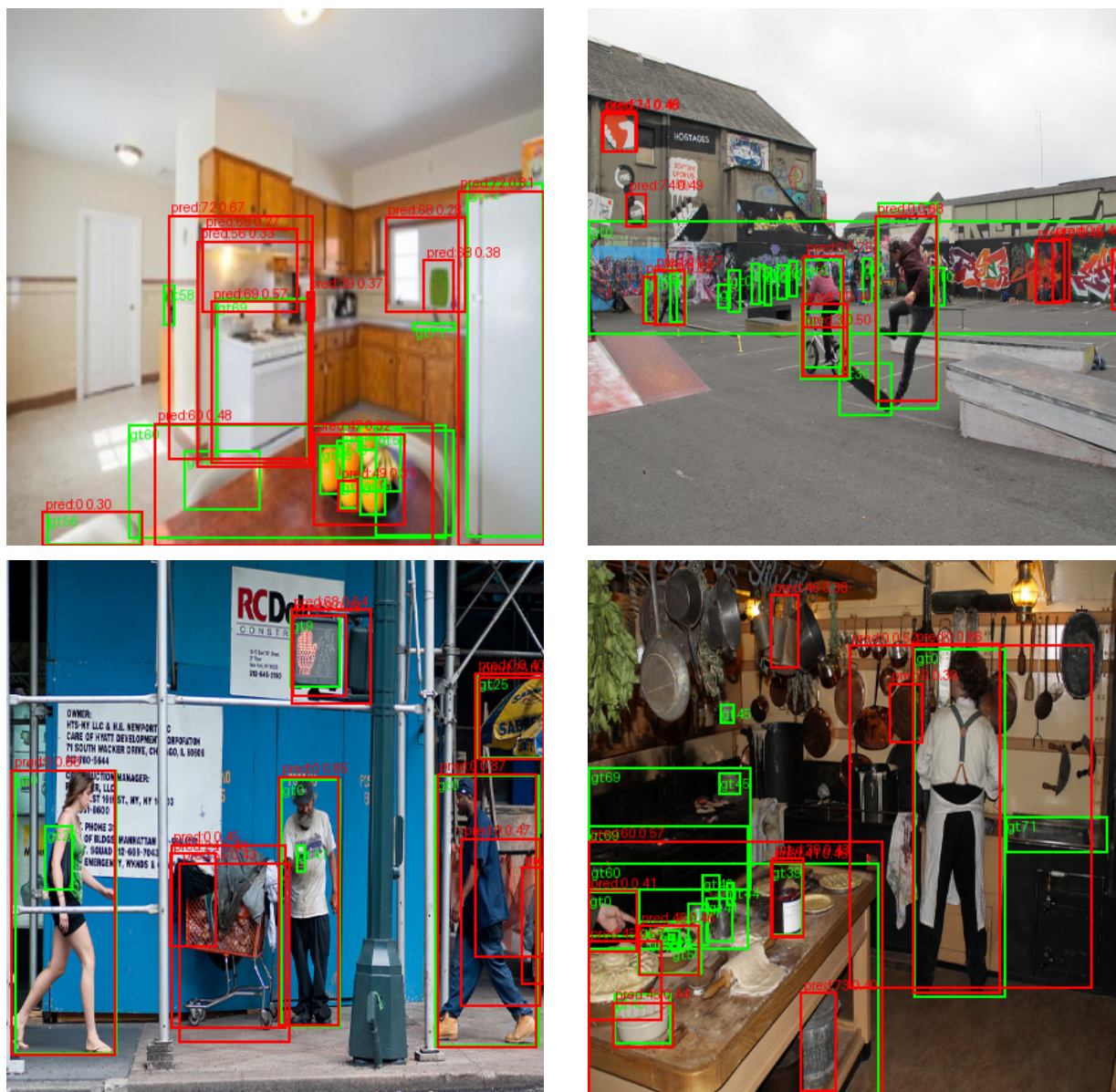


Figure 8: 本轮实验默认保存的4 张验证集检测可视化

为了最终汇报，我们额外保存12张扩展展示图作为更完整的结果展示。该组图与上面的4张图使用同一套展示参数：score threshold 0.25、top-k 12、score $\alpha = 2.0, \beta = 1.0$ 。这样不同展示图之间的框数量和置信度筛选规则保持一致，便于直接比较视觉效果。



Figure 9: 本轮实验用于最终展示的12 张扩展展示图

9.2 实验流程与配置文件

该实验使用的配置文件快照为:

`logs/records/dual_scale_three_box_coco_only_noobj1.416_ddp_20260430_124818/config.toml`.

原始配置来自:

`configs/dual_scale_three_box_coco_only_noobj1.416.toml`.

Table 5: 本轮实验的关键配置

类别	配置
训练集	COCO2017 train, 118287 张
验证集	COCO2017 val, 5000 张
数据格式	packed .pt, chunk size 1024, cache 4 chunks
输入尺寸	416 × 416
数据增广	本轮无随机增广; 执行resize 与box 同步缩放
模型	P4/P5 双尺度, decoupled head, FPN/PAN Lite
Anchor	原始P4/P5 anchors
Loss	GIoU + objectness BCE + quality-aware cls BCE
Assignment	dynamic_cost, dynamic_topk=2
优化器	AdamW, lr 0.001, weight decay 0.0001
学习率调度	cosine, min lr 1×10^{-5}
训练轮数	30 epoch
并行方式	8 GPU DDP, NCCL backend
Batch size	每卡16, 全局有效batch size 128
DataLoader	总num_workers=64, 每rank 8 workers

训练流程可以概括为: 先读取配置文件并构建COCO-only dataset; 随后校验packed .pt index 与chunk 文件是否完整; 再构建P4/P5 双尺度模型、原始P4/P5 anchors、YOLOLoss、AdamW optimizer 和cosine scheduler; 最后通过torchrun 启动8 进程DDP 训练。每个epoch 后进行validation, 并按best validation loss 保存best.pth。

本轮训练从零开始, 没有从已有checkpoint resume。实验完成状态为completed。训练总耗时约24117.2 秒, 验证总耗时约665.4 秒, 训练加验证合计约6.884 小时。单个训练epoch 平均约803.9 秒, 即约13.40 分钟; 单个validation epoch 平均约22.2 秒。每个epoch 有925 个optimizer steps, 训练全量样本约118288, 验证样本为5000。

9.3 训练过程中保存的检测与监控指标

本轮模型profile 的正式记录来自COCO eval JSON。训练过程中的loss、learning rate、assignment 指标和定位质量指标以TensorBoard event 为原始记录来源, 对应目录为:

runs/dual_scale_three_box_coco_only_noobj1_416_ddp_20260430_124818.

模型参数量为36.53M, 总FLOPs 为78.38 GFLOPs。测速配置为batch size 1, warmup 20 次, measure 100 次; 记录到的平均推理耗时为3.53 ms/batch, 吞吐约283.32 images/s。这里的FLOPs 统计主要来自Conv2d, Linear FLOPs 为0。

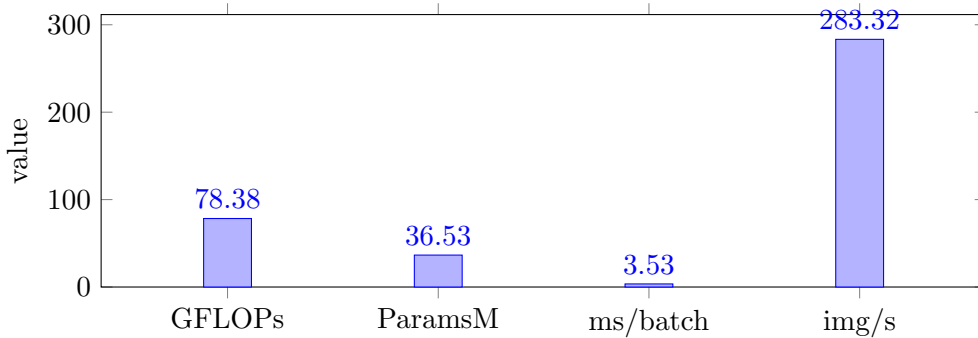


Figure 10: 本轮实验模型复杂度与推理profile。不同柱子的单位不同, 图中用于集中展示profile 量级。

Loss 下降曲线如下。该图由TensorBoard 中的epoch 级标量导出，使用的tag 包括`loss/total_epoch`、`val/total_epoch`和`val/box_epoch`。训练集loss 从7.2061 降到3.7613，验证集loss 从6.8091 降到4.8544；best validation loss 出现在第23 轮，为4.7799。训练loss 持续下降，而验证loss 在后期趋于平台并略有回升，说明模型已经进入明显的收敛后段。

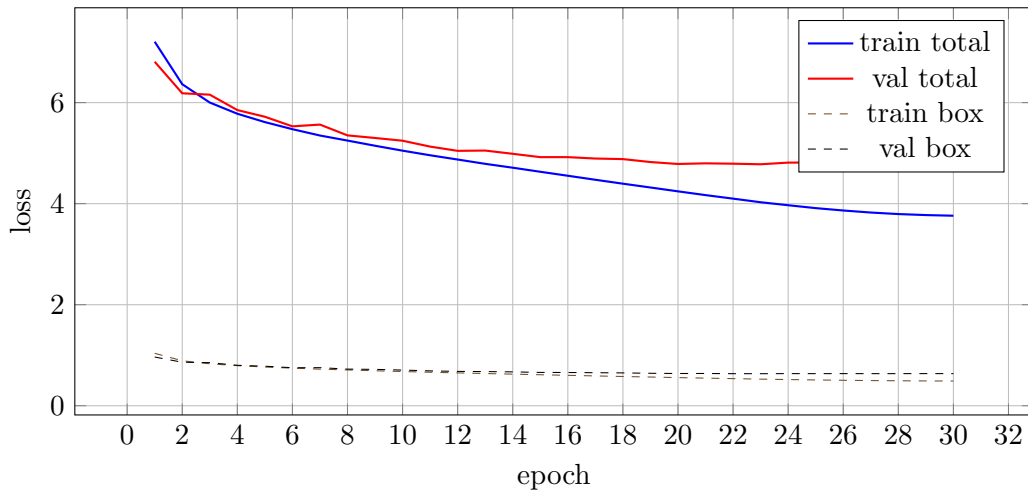


Figure 11: 训练集与验证集loss 下降曲线

TensorBoard 同时保存了step 级训练标量。由于DDP 中step 级TensorBoard 当前记录的是rank0 局部batch 的loss，因此它更适合观察训练过程是否发散、是否存在异常尖峰，而不应被解释为8 卡全局平均step loss。下图使用`rank0_loss/*_step` 标量下采样绘制，用于展示step 级训练波动。

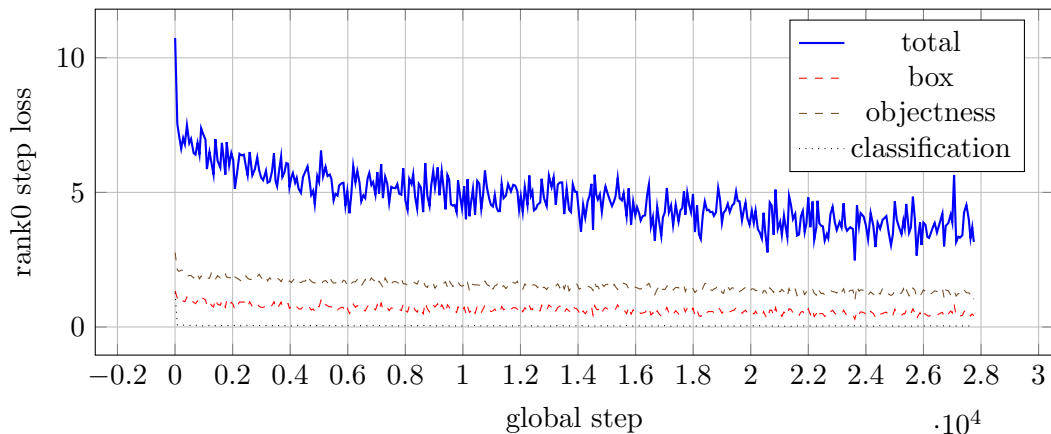


Figure 12: TensorBoard step 级rank0 loss 曲线。图中数据来自`rank0_loss/*_step`，为版面展示做了下采样。

下面的图展示epoch-to-epoch 的loss 改变量。它同样由TensorBoard epoch scalar 计算得到，用于监控loss 下降速度，也就是“loss 曲线的一阶变化”。需要注意，这不是参数梯度范数；本轮实验没有保存真实的parameter grad norm。因此该图只能说明loss 收敛速度，而不能替代梯度范数监控。后续若要做真正的梯度监控，需要在训练脚本中额外记录`grad_norm`、是否出现NaN/Inf、以及必要时的gradient clipping 状态。

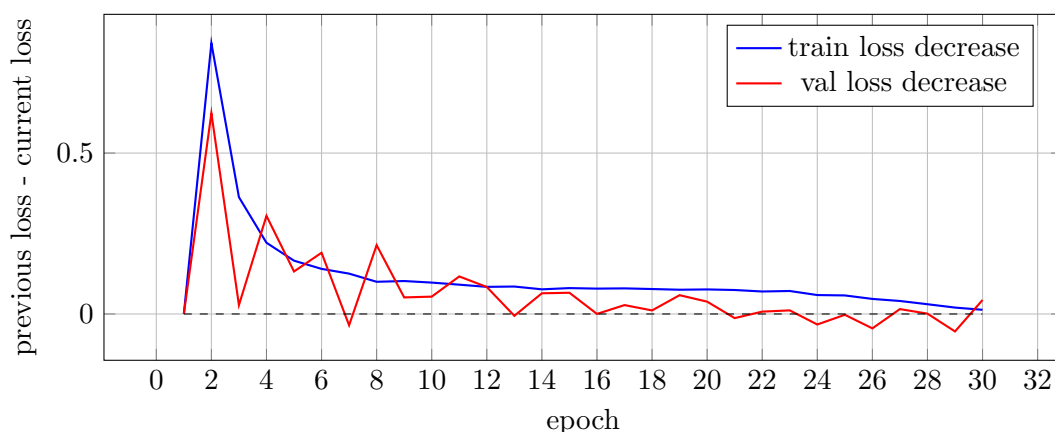


Figure 13: Loss 下降速度监控。负值表示该epoch 的损失相比上一轮上升。

除了loss，本轮TensorBoard 还记录了assignment 与定位相关指标。正样本平均GIoU 从训练第1 轮的0.4814 提升到第30 轮的0.7554；验证mean GIoU 从0.5179 提升到0.6823。每图正样本槽位数约从26-27 提升并稳定在28 左右，说明dynamic assignment 持续给每张图分配了稳定数量的正样本。

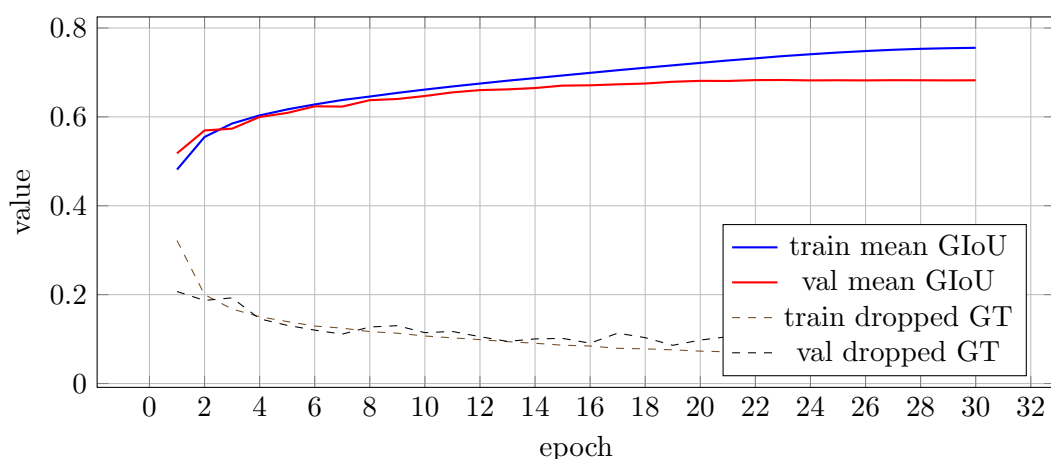


Figure 14: 定位与assignment 健康度监控

9.4 正式COCO 指标与实际意义

COCO AP 使用多个IoU 阈值综合评价检测质量。AP 是0.50 : 0.05 : 0.95 共10 个IoU 阈值上的平均precision-recall 面积；AP50 只看IoU=0.50，主要反映粗定位下的召回和分类能力；AP75 更强调较准确定位。APS、APM、APL 分别对应small、medium、large 三种目标尺寸。AR1、AR10、AR100 表示每张图最多保留1、10、100 个预测时的平均召回能力。

Table 6: COCO 指标含义与本轮结果

指标	计算规则/含义	本轮结果
AP	IoU 0.50:0.95 平均AP	0.1488
AP50	IoU=0.50 的AP	0.2487
AP75	IoU=0.75 的AP	0.1553
AP small	小目标AP	0.056
AP medium	中目标AP	0.159
AP large	大目标AP	0.214
AR1	每图最多1 个预测的平均召回	0.1889
AR10	每图最多10 个预测的平均召回	0.2898
AR100	每图最多100 个预测的平均召回	0.3023
预测数量	全验证集最终预测框数	260274
预测/图	平均每张图预测框数量	52.0548

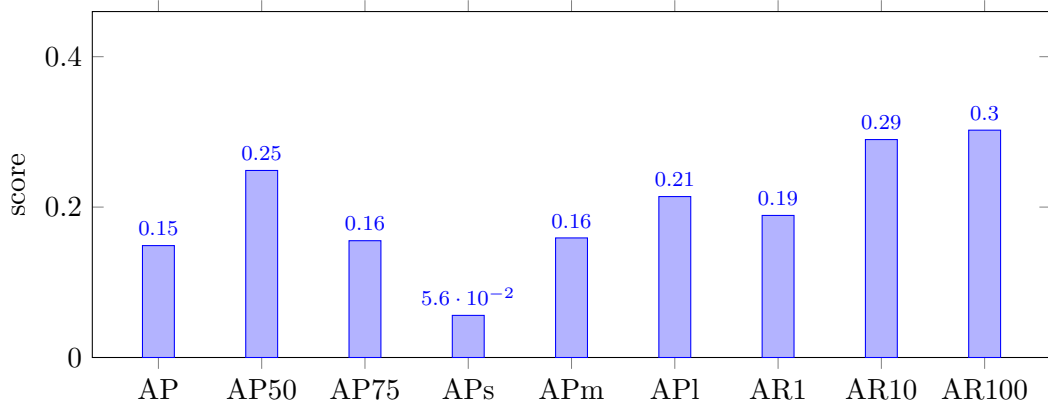


Figure 15: 本轮实验COCO AP/AR 指标可视化

从结果解释上看，本轮模型已经具备基本检测能力，但距离成熟YOLO 检测器仍有明显差距。AP50 高于AP75，说明模型在“粗略找到目标”方面优于“高质量定位”；AP small 明显低于AP medium 和AP large，说明当前P4/P5 双尺度结构对小目标不够友好，这也是后续增加P3 的直接动机。AR100 为0.3023，说明即便每张图允许保留较多候选，召回仍然有限，因此后续改进不能只靠降低score threshold，还需要从分辨率、P3 特征、小目标anchor 和assignment 上继续提升候选质量。

本轮默认评估配置为score threshold 0.05、top-k 100、NMS IoU 0.5、score $\alpha = 2.0, \beta = 1.0$ 。后续非训练sweep 显示，在不重新训练的情况下，在500 张诊断子集上，score threshold 0.01 可提升AP 到约0.1800，NMS IoU 0.7 可将AR100 提高到约0.3361。这说明当前模型仍有一部分召回受到推理阈值和NMS 设置影响，但正式结论仍以全量COCO eval 为准，根本改进仍需要训练方向的结构和数据策略调整。